

# Spenny Piggy – Risk, Payments & Admin Controls (Implementation Doc)

This document explains what has been implemented across the two repos, why each feature exists, who uses it, and how the full flow works end-to-end.

## Repositories

- Main site (user/creator/gifter flows): [spennypiggy.co](https://spennypiggy.co)
- Admin site (founder/admin operations): [admin.spennypiggy.co](https://admin.spennypiggy.co)

Both apps read/write the same underlying tables for the risk engine (payments ledger, identities, rollups, platform state, disputes, etc.).

## Primary Goals

- Block fraud and chargeback patterns before Stripe is hit when possible
- Reduce Stripe risk signals (excessive failures, card testing patterns)
- Give admins full control over thresholds/limits from the admin dashboard
- Keep creators safe: reserves + review holds + payout delays based on risk
- Provide auditability: logs, exports, and traceable decisions

## Roles (Who Uses What)

- Supporter/Gifter: pays creators (cart checkout, tip jar, etc.)
- Creator: receives support; may be restricted based on subscription/activity/risk
- Admin/Founder: tunes risk settings, monitors action queue, runs payouts, exports audit pack

## Key Concepts

### 1) Risk Identity

Purpose: link payment attempts to a stable “person/device” so limits cannot be bypassed by changing email.

Resolution order:

1. Card fingerprint (when available)
2. Device ID (frontend generated)
3. Email
4. IP

Main code: [spennypiggy.co/app/Services/Risk/RiskIdentityService.php](https://spennypiggy.co/app/Services/Risk/RiskIdentityService.php)

## 2) Payments Ledger (Risk Engine “payments” table)

Purpose: a platform-side ledger used for limits, velocity rules, review holds, payout selection, and reporting.

Key statuses used:

- **initiated**: checkout created / attempt started
- **step\_up**: STEP\_UP triggered (OTP required)
- **succeeded**: successful payment
- **review\_hold**: captured but held for payout safety
- **refunded, disputed**: negative adjustments

This ledger is what the Risk Engine uses to compute “how much was spent in the last hour/day/week”.

## 3) Identity Rollups

Purpose: fast counters for spend and attempt frequency.

Examples:

- **spend\_1h, spend\_24h, spend\_7d**
- **payment\_count\_10m**
- **creators\_paid\_24h, creators\_paid\_48h**

Main code: [spennypiggy.co/app/Services/Risk/IdentityRollupService.php](https://spennypiggy.co/app/Services/Risk/IdentityRollupService.php)

Important behavior:

- Spend windows include **initiated** to prevent “open multiple checkouts then exceed cap” bypass.

## 4) Platform Risk State

Purpose: automatic “brakes” on the whole platform when risk spikes.

States:

- NORMAL
- CAUTION
- THROTTLE
- FREEZE

The platform state influences effective limits and creator onboarding.

Main code:

- State monitor command:  
`spennypiggy.co/app/Console/Commands/MonitorPlatformRiskState.php`
- Effective limits service:  
`spennypiggy.co/app/Services/Risk/EffectiveLimitsService.php`

## Admin Configurable Settings (Risk Configuration)

Admin page: `admin.spennypiggy.co` → `/risk/configuration`

All values are stored in `risk_settings` as JSON and applied at runtime.

### Global Limits (Hard Cap)

Purpose: platform “maximum” caps that apply across all states.

Keys:

- `max_spend_1h`, `max_spend_24h`, `max_spend_7d`
- `max_creators_per_day`
- `guest_allowed`

### State Limits (NORMAL/CAUTION/THROTTLE/FREEZE)

Purpose: per-state limits; effective limits are the min(state limit, global limit).

Keys per state:

- spend caps (1h/24h/7d)
- guest allowed
- cooldown minutes
- step-up threshold
- review-hold threshold
- max new creators (24h)

## **Supporter Rules**

Purpose: velocity rules + high-value triggers.

- Velocity window (currently rollup window is fixed 10 minutes)
- STEP\_UP count threshold
- COOLDOWN count threshold
- Single transaction STEP\_UP threshold

## **Creator Rules**

Purpose: protect very new creators.

- New creator age in days
- New creator daily volume cap

## **High Velocity Spend Rules**

Purpose: 2-hour high-spend patterns.

- 2h STEP\_UP threshold
- 2h REVIEW\_HOLD threshold
- force 3DS toggle

## **Cross Creator Rules**

Purpose: prevent “whale hopping”.

- 48h spend trigger
- min creators paid in 48h
- restriction duration hours

## **Platform State Triggers**

Purpose: automatic CAUTION/THROTTLE/FREEZE rules.

- daily GMV multipliers
- weekly GMV multiplier
- platform dispute rate freeze threshold
- creator dispute cluster threshold + count

## **Payment Entry Points (Where Risk Engine Is Applied)**

**Cart Checkout (supporter checkout)**

- Controller:  
`spennypiggy.co/app/Http/Controllers/Auth/CheckoutController.php`
- Risk evaluation happens before creating Stripe Checkout Session.
- Ledger entry is recorded with `stripe_session_id`.

## Tip Jar Support

- Controller:  
`spennypiggy.co/app/Http/Controllers/Auth/StripeController.php (tipToJar)`
- Risk evaluation is performed before Stripe.
- STEP\_UP (OTP + CONFIRM) is supported.
- Ledger entry recorded with `stripe_session_id`.

## Webhook mapping (Checkout Session → ledger)

- Controller:  
`spennypiggy.co/app/Http/Controllers/StripeWebhookController.php`
- On `checkout.session.completed`, it maps the Stripe session id to the ledger payment and sets status (keeps `review_hold` if flagged).

## Friendly Payment Block Messages

Problem solved: generic “creator temporarily unavailable” made it hard to debug and confused users.

Now the user sees a friendly, specific reason for common blocks:

- Creator subscription inactive (“Wishlist plan not active”)
- Creator needs to update content (“temporarily unavailable while updating page”)
- Account status issue (“account status issue”)

Mapper service:

`spennypiggy.co/app/Services/CreatorAvailabilityMessageService.php`

## Stripe Webhooks: Idempotency + Processing Safety

Webhook table: `stripe_webhook_status`

Behavior:

- If event already processed → ignore

- If processing recently → ignore
- If failed/stale → retry allowed
- After successful handling → mark processed with timestamp

Main code:

[spennypiggy.co/app/Http/Controllers/StripeWebhookController.php](https://spennypiggy.co/app/Http/Controllers/StripeWebhookController.php)

## Early Fraud Warnings (EFW)

When Stripe sends `early_fraud_warning.created`:

- Record `early_fraud_warnings` row
- Link to ledger payment (if possible)
- Add audit log entry
- Notify creator with refund recommendation

Main code:

[spennypiggy.co/app/Http/Controllers/StripeWebhookController.php](https://spennypiggy.co/app/Http/Controllers/StripeWebhookController.php)  
(`handleEarlyFraudWarningCreated`)

## Payout Engine (Friday Payouts)

Purpose:

- Pay creators safely after applying reserve % and payout delays
- Exclude review holds
- Subtract refunds/disputes
- Carry negative balances forward

Key tables:

- `payout_runs` (preview + executed)
- `creator_metrics` includes: `reserve_percent`, `payout_delay_days`, `negative_balance_minor`
- `payments` ledger uses `payout_run_id` to mark accounted entries

Main service:

- [spennypiggy.co/app/Services/Risk/PayoutService.php](https://spennypiggy.co/app/Services/Risk/PayoutService.php)

Admin controls:

- Admin dashboard buttons under `/risk` (Preview / Execute)

- Admin endpoints in `admin.spennypiggy.co/app/Http/Controllers/Admin/RiskController.php`

## Audit Pack Export (ZIP)

Purpose: one click generates a ZIP containing CSVs for audits (Stripe review / compliance).

Admin endpoint:

- `admin.spennypiggy.co → /risk/export/audit-pack`

Includes CSVs:

- disputes, refunds, platform states
- confirmation logs
- creator metrics
- top identities by spend
- webhook logs (if table exists)
- platform summary

## Spend Visibility for Gifter

On `/history` (support history), gifters can see:

- last 1 hour spend
- last 24 hours spend
- last 7 days spend
- their current limits

Display currency behavior:

- uses selected currency cookie when available
- conversion uses Currency rates and ISOdigits rounding (same approach as finance)

## Currency & Earnings Correctness

Fixes applied:

- Finance dashboard summary now uses correct columns:
  - gross income = `gross_amount`
  - net income = `net_amount`

- Summary conversion uses Currency ISODigits rounding (consistent with finance/tax conversion style).
- Finance uses the selected display currency cookie when set.

Main code:

- `spennypiggy.co/app/Services/FinancialService.php`
- `spennypiggy.co/app/Http/Controllers/CreatorFinancialController.php`

## Operational Commands

- Risk go-live checks:
  - `php artisan risk:go-live-check`
- Simulate repeated risk decisions (no Stripe calls):
  - `php artisan risk:simulate-payments --creator=<uuid> --amount=1000 --count=5 --device=device-1 --email=a@b.com`

## Admin Dashboard Enhancements

Risk dashboard (`admin.spennypiggy.co` → `/risk`) now includes:

- Summary cards (GMV, dispute rate, review-hold, exposure estimate, cooldown identities)
- Action Queue (disputes + EFWs)
- Payout preview/execute controls
- Audit pack download

## Known Constraints / Notes

- Velocity “window minutes” is displayed in admin settings, but current rollup window uses a fixed 10 minutes field (`payment_count_10m`). Making the window fully dynamic requires rollup schema changes.
- Currency conversion uses GBP as a base currency using stored `conversion_rate` fields.